



PF

Final Project – Tetris Game

Submitted By:

- **M Ehtasham Ali (572)**
 - **Jahanzaib (558)**
 - **Haseeb Najam (534)**
 - **Amna Tariq (524)**
 - **Abid Shafique (502)**

Submitted To:

- **Sir Muzammil Yousaf & Mam Rabia Sana**

Department of Computer Science UET(Narowal)

Project Report

Project Report

Tetris Game in C++

This project report outlines the development of a simple Tetris game implemented in C++ using basic programming constructs. The objective of the project was to enhance programming skills and apply knowledge of algorithms, and basic game development principles. The game features core Tetris mechanics, including block rotation, movement, and line clearing, and is implemented using a console-based interface. This report discusses the methodology, code structure, results, and potential improvements.

Table of Contents:

1. Introduction
2. Objectives
3. Methodology
4. Implementation Details
5. Results and Discussion
6. Conclusion and Future Work
7. References

1. Introduction:

Tetris is a popular puzzle game that involves arranging falling blocks of different shapes to form complete rows, which are then cleared. The aim of this project was to create a simple console-based version of Tetris using the C++ programming language. The project provided an opportunity to apply theoretical programming concepts to a practical application and gain hands-on experience in game development.

2. Objectives:

The primary objectives of this project were:

1. To develop a functional Tetris game using C++.
2. To implement game mechanics such as block rotation, movement, and line clearing.
3. To enhance understanding of programming concepts, including arrays, loops, functions.
4. To create a user-friendly console interface for gameplay.

3. Methodology:

- **Planning:**

The project was divided into the following stages:

- Understanding Tetris gameplay mechanics.
- Designing the program structure.
- Implementing the game logic and user interface.
- Testing and debugging.

- **Tools and Technologies:**

- **Programming Language:** C++
- **Development Environment:** [IDE or Text Editor Used, e.g., Visual Studio Code, Dev-C++]

4. Implementation Details:

- **Program Structure:**

The program is divided into the following components:

- **Playfield:** A 15x10 grid represented as a Boolean array to track the game state.
- **Tetramines:** Predefined shapes encoded as binary 3x3 maps, allowing for efficient representation and manipulation.
- **Game Logic:** Functions to handle:
 - Block movement and collision detection.
 - Line clearing and scoring.
 - Game termination conditions.
- **User Input:** Captures player commands (e.g., rotate, move left/right, increase speed).
- **Display:** Uses console output to represent the playfield and game status.

- **Key Functions and Algorithms:**

- **Initialize Board ():** Initializes the game board and sets up boundaries.
- **Spawn Tetramine ():** Generates a random tetramine.
- **Move Tetramine(direction):** Handles horizontal movement.
- **Rotate Tetramine ():** Changes the orientation of the tetramine.
- **Clear Lines ():** Detects and clears full rows, awarding points.
- **Display Board ():** Prints the playfield to the console.

5. Main Game Loop:

- Initialize the game board and set starting parameters.
- Enter the loop:

- Update the tetramine position.
- Display the game state.
- Handle user input and apply transformations.
- Detect collisions and resolve them.
- Check for and clear completed lines.
- End the game if conditions are met (e.g., blocks reach the top of the playfield).

6. Code Snippet Overview:

```

bool playfield [150] = {}; // Represents the game board.
int figures [16] = {56, 146, 56, 146, 58, 178, 184, 154, 312, 210, 57, 150, 27, 27, 27, 27};
int offset = 1, points = 0;
bool running = true;

while (running) {
    // Game logic: input handling, collision detection, block movement, line clearing.
    SetConsoleCursorPosition (GetStdHandle (STD_OUTPUT_HANDLE), coords);
    Display Board ();
    // Process collisions and update game state.
    if (! collision) offset += 10; // Move tetramine down.
    else spawn Tetramine (); // Reset on collision.
}

```

7. Results and Discussion:

The Tetris game was successfully implemented with the following features:

- Functional tetramines with rotation and movement mechanics.
- Row clearing and scoring system.
- Console-based visual representation of the playfield.

Challenges:

- Implementing collision detection for rotated tetramines.
- Handling game speed regulation using delays.

Limitations:

- Console-only interface without advanced graphics.
- Lack of a pause or save feature.

8. Conclusion and Future Work:

This project demonstrated the successful implementation of a basic Tetris game using C++. It allowed for practical application of programming concepts and enhanced problem-solving skills. Future improvements include:

1. Adding a graphical interface using libraries like SFML or SDL.
2. Introducing advanced features such as levels, speed variation, and power-ups.
3. Implementing sound effects and music for an engaging experience.
4. Including multiplayer functionality for competitive gameplay.

Code C++

```
#include <iostream>

#include <windows.h> //needed for setting text cursor position and Sleep()

using namespace std;

int main()
{
    bool playfield[150] = {}; // playfield is 15x10
    int figures[16] = {56, 146, 56, 146, 58, 178, 184, 154, 312, 210, 57, 150, 27, 27, 27, 27};

    /* 4 blocks in 4 orientations, encoded as binary 3x3 maps (9 bits)
    So for example L-block is 010 010 110 = 150
    block position (1 row=10) */

    int offset = 1;

    bool running = true; // flag: game is running?

    int time = 0;

    int key_delay = 0;

    int points = 0;

    bool increased_speed = false; // increased block speed flag

    int direction = 0; // block orientation

    int figure_number = 0; // block type
```

```

bool collision = false;

int new_position[2] = {}; // new position after applying controls, to test if input will cause a
collision

COORD coords = {0, 0}; // for setting console pos. to beginning

for (int i = 0; i < 150; i++)
{
    playfield[i] = (i % 10 == 0 || i % 10 == 8 || i > 140); // draw the edge around playfield
}

while (running) // main loop
{
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), coords);
    cout << "points:" << points << "    " << "\n"; // set to 0,0 , output points
    time++;
    offset += 10 * (time % (10 - 9 * increased_speed) == 0);
    if (key_delay > 0)
    {
        key_delay--; // advance time, move block down, reduce key delay (key_delay)
    }

    collision = false; // no collision?
    for (int i = 0; i < 9; i++)
    {

```

```

        collision += playfield[offset + i % 3 + i / 3 * 10 + 10] * ((figures[figure_number * 4 +
direction] & 1 << i) > 0); // collision>0 when there is something below active block

        playfield[offset + i % 3 + i / 3 * 10] += ((figures[figure_number * 4 + direction] & 1 <<
i) > 0); // draw the block
    }
    for (int i = 0; i < 150; i++)
    {
        cout << (char)(32 + 3 * playfield[i] - 22 * (i % 10 == 9)); // draw the play field:
32(space) or 35(#) or 10(newline)
    }
    if (!collision)
    {
        for (int i = 0; i < 9; i++)
        {
            playfield[offset + i % 3 + i / 3 * 10] -= ((figures[figure_number * 4 + direction] & 1 <<
i) > 0); // there was no collision?
        }
    }
    else // If so, erase the block, it will be drawn lower in next loop
    {
        if (offset < 10)
        {
            running = false;
        }
        offset = 1;
        figure_number = rand() % 4;
        direction = 0;
    }

```

```

increased_speed = false;

    /* collision happened?
        put the block and create a new one.
        We are at the top? Game over. */
}
Sleep(50); // bad way of regulating speed
new_position[0] = offset;
new_position[1] = direction; // save the position and angle of the block
if ((GetKeyState(VK_ESCAPE) & 0x8000))
{
    running = false;
}
if ((GetKeyState(VK_RIGHT) & 0x8000) && (offset % 10 < 6)) // right
{
    offset++;
}
if ((GetKeyState(VK_LEFT) & 0x8000) && (offset % 10 > 0)) // left
{
    offset--;
}
if ((GetKeyState(VK_DOWN) & 0x8000)) // down - speedup
{
    increased_speed = true;
}

```

```

else
{
    increased_speed = false;
}

if ((GetKeyState(VK_UP) & 0x8000) && key_delay == 0) // up - rotation
{
    direction = (direction + 1) % 4;
    key_delay = 4;
}

collision = false;
for (int i = 0; i < 9; i++)
{
    if (playfield[offset + i % 3 + i / 3 * 10] + ((figures[figure_number * 4 + direction] & 1 <<
i) > 0) > 1)
    {
        collision = true; // controls cause the block to collide with something?
    }
}

if (collision) // collision happened - reset position and angle (e.g. move was impossible)
{
    offset = new_position[0];
    direction = new_position[1];
}

```

```
for (int j = 0; j < 14; j++) // go through field rows
{
    collision = true;
    for (int i = 1; i < 8; i++)
    {

if (playfield[i + j * 10] == 0)
    {
        collision = false; // test if we have a whole line
    }
    }

if (collision) // whole line? remove blocks, add points...
{
    for (int i = 1; i < 8; i++)
    {
        playfield[i + j * 10] = 0;
    }
    points++;

    for (int j2 = j; j2 > 0; j2--)
    {
```

```

for (int i = 1; i < 8; i++) // Move everything above line one block down
{
    playfield[i + j2 * 10] = playfield[i + j2 * 10 - 10];
}
}
}
}
}

cout<<endl;
cout<<endl;
cout<<"Game Over! <<>> Your Points = "<<points;
return 0;
}

```

Output Result

```

C:\Users\hp\Desktop\tetriss.e
points:1
#      #
#      #
#      #
#      #
####  #
#      #
#      #
#      #
#      #
####  #
##     #
##     # #
###  ####
###  ####
#####

```

